

Term Project – Documentation

Blutspende-Verwaltungssystem: a blood donation management system

Dozent: Stephan Bökelmann sboekermann@ep1.rub.de Technische Hochschule Georg Agricola, Bochum

Author: Amen Dabbech

Module: Introduction to Database Management Systems · THGA Bochum

Stack: PostgreSQL · FastAPI · Docker Compose · Python CLI (.deb)

Repository: https://github.com/amendabbech/DBMS_10

Inhaltsverzeichnis

1	Introduction and motivation	1
2	Data model	2
2.1	Relational schema	2
3	Database (PostgreSQL)	2
4	REST API (FastAPI)	3
5	Frontend (Python CLI, packaged as .deb)	4
6	Operation and deployment	4
7	Reflection	5

1 Introduction and motivation

Blood donation centers often coordinate donors and appointments manually or with simple spreadsheets. This makes it hard to enforce basic safety rules, such as the mandatory waiting period between two donations by the same donor, and to answer questions such as “*which donors are currently eligible to donate again?*” without manual checking.

Blutspende-Verwaltungssystem replaces this manual process with a proper relational system. It manages **Spender** (donors), **Spendezentren** (donation centers) and **Spenden** (donations), exposes the data through a REST API, and offers a small command-line frontend for staff.

Usage scenario. A donor arrives at a donation center to give blood. Staff open the client, look up the donor, and register a new donation with date, blood type and amount. The system automatically rejects the donation if the donor’s last donation was less than 56 days

ago, enforcing the safety rule without any manual date calculation.

2 Data model

The domain has three entity types. A **Spender** can give many **Spende** over time at different **Spendezentren**, and a **Spendezentrum** can host donations from many donors – an N:M relationship between **Spender** and **Spendezentrum** that is resolved by the **Spende** entity.

ER model (textual).

- **Spender** (1) – gives – (N) **Spende**
- **Spendezentrum** (1) – hosts – (N) **Spende**

Thus **Spender** N:M **Spendezentrum**, resolved through **Spende**.

2.1 Relational schema

Derived from the ER model, with primary keys underlined and foreign keys in italics:

- **spender**(spender_id, vorname, nachname, geburtsdatum, blutgruppe, email)
- **spendezentrum**(zentrum_id, name, adresse, stadt)
- **spende**(spende_id, *spender_id*, *zentrum_id*, spende_datum, menge_ml)

Normalisation. Every non-key attribute depends only on the whole primary key of its table, and there are no transitive dependencies: a donor's email lives only in **spender**, a center's address only in **spendezentrum**, and a donation references donors and centers by their keys rather than duplicating their data. The schema is therefore in **3NF**.

3 Database (PostgreSQL)

The schema is created by an init script that PostgreSQL runs on first start, via a Docker volume mount. In addition to the three tables, a **trigger** enforces the core business rule of the domain: a donor may not donate again within 56 days of their last donation.

Listing 1: schema.sql (excerpt) – tables and 56-day trigger

```
1 CREATE TABLE spender (
2     spender_id SERIAL PRIMARY KEY,
3     vorname VARCHAR(100) NOT NULL,
4     nachname VARCHAR(100) NOT NULL,
5     geburtsdatum DATE NOT NULL,
6     blutgruppe VARCHAR(3) NOT NULL,
7     email VARCHAR(255) UNIQUE NOT NULL
8 );
9
10 CREATE TABLE spendezentrum (
11     zentrum_id SERIAL PRIMARY KEY,
12     name VARCHAR(150) NOT NULL,
13     adresse VARCHAR(255) NOT NULL,
14     stadt VARCHAR(100) NOT NULL
15 );
16
17 CREATE TABLE spende (
18     spende_id SERIAL PRIMARY KEY,
```

```

19     spender_id INTEGER NOT NULL REFERENCES spender(spender_id),
20     zentrum_id INTEGER NOT NULL REFERENCES spendezentrum(zentrum_id),
21     spende_datum DATE NOT NULL,
22     menge_ml INTEGER NOT NULL CHECK (menge_ml > 0)
23 );
24
25 CREATE OR REPLACE FUNCTION pruefe_spendeabstand()
26 RETURNS TRIGGER AS $$
27 BEGIN
28     IF EXISTS (
29         SELECT 1 FROM spende
30         WHERE spender_id = NEW.spender_id
31         AND spende_datum > NEW.spende_datum - INTERVAL '56 days'
32         AND spende_datum < NEW.spende_datum + INTERVAL '56 days'
33     ) THEN
34         RAISE EXCEPTION 'Mindestabstand von 56 Tagen nicht eingehalten';
35     END IF;
36     RETURN NEW;
37 END;
38 $$ LANGUAGE plpgsql;
39
40 CREATE TRIGGER trg_spendeabstand
41 BEFORE INSERT ON spende
42 FOR EACH ROW
43 EXECUTE FUNCTION pruefe_spendeabstand();

```

4 REST API (FastAPI)

The API is the only component that talks to the database. Read endpoints are public; write endpoints require the header X-API-Key.

Method	Path	Purpose	Key
GET	/spender	list all donors	–
POST	/spender	register a new donor	yes
GET	/spendezentrum	list all donation centers	–
POST	/spendezentrum	register a new center	yes
GET	/spende	list all donations	–
POST	/spende	register a new donation	yes

The POST /spende endpoint demonstrates how a database-level **constraint** (the 56-day trigger) is surfaced as a clean API error instead of a raw database exception:

Listing 2: POST /spende – catching the trigger’s exception

```

1 @app.post("/spende")
2 def create_spende(spende: SpendeIn, auth: None = Depends(check_api_key))
3 :
4     conn = get_conn()
5     cur = conn.cursor(cursor_factory=RealDictCursor)
6     try:
7         cur.execute(
8             "INSERT INTO spende (spender_id, zentrum_id, spende_datum,
9             menge_ml) "

```

```

8         "VALUES (%s, %s, %s, %s) RETURNING *",
9         (spende.spender_id, spende.zentrum_id,
10         spende.spende_datum, spende.menge_ml)
11     )
12     new_row = cur.fetchone()
13     conn.commit()
14     return new_row
15 except psycopg2.Error as e:
16     conn.rollback()
17     raise HTTPException(status_code=400, detail=str(e))
18 finally:
19     cur.close()
20     conn.close()

```

The write path is guarded by a simple dependency that checks the API key:

Listing 3: API-key guard

```

1 API_KEY = "geheimsschlüssel123"
2
3 def check_api_key(x_api_key: str = Header(...)):
4     if x_api_key != API_KEY:
5         raise HTTPException(status_code=401, detail="Ungültiger API-Key")

```

5 Frontend (Python CLI, packaged as .deb)

The frontend is a small command-line client with a numbered menu offering six actions: list and create Spender, Spendezentrum and Spende. It never touches the database directly – it only calls the API via HTTP.

Packaging. The client script is wrapped into a Debian package with a minimal `DEBIAN/control` file and installed as a system-wide command:

Listing 4: Building and installing the .deb

```

1 mkdir -p blutspende-client/DEBIAN
2 mkdir -p blutspende-client/usr/local/bin
3 cp client.py blutspende-client/usr/local/bin/blutspende-client
4 chmod +x blutspende-client/usr/local/bin/blutspende-client
5 dpkg-deb --build blutspende-client
6 sudo dpkg -i blutspende-client.deb

```

After installation, the client is available system-wide as the `blutspende-client` command.

6 Operation and deployment

The backend runs as a PostgreSQL container orchestrated by Docker Compose. The schema is loaded automatically from a mounted SQL file on first start.

Listing 5: docker-compose.yml (excerpt)

```

1 services:
2     db:
3         image: postgres:16
4         environment:
5             POSTGRES_USER: dbms10

```

```
6     POSTGRES_PASSWORD: dbms10pass
7     POSTGRES_DB: blutspende
8     ports:
9       - "5433:5432"
10    volumes:
11      - ./system/db/schema.sql:/docker-entrypoint-initdb.d/schema.sql
12      - pgdata:/var/lib/postgresql/data
13
14 volumes:
15   pgdata:
```

Deployment steps.

1. `docker-compose up -d` – start the database.
2. `uvicorn main:app -host 0.0.0.0 -port 8000` – start the API.
3. `sudo dpkg -i blutspende-client.deb` – install the frontend.
4. Run `blutspende-client` to interact with the system.

7 Reflection

The separation into database, API and CLI client made the 56-day business rule easy to verify independently at each layer: first directly in PostgreSQL via `psql`, then through the API with `curl`, and finally through the client itself. The trickiest part was choosing where to enforce the rule – a database trigger was chosen over an application-level check so that the constraint holds regardless of which client inserts data. Given more time, a dedicated endpoint to check a donor's eligibility date in advance (instead of only rejecting at insert time) would improve the user experience.